Algorytmy Metaheurystyczne - Laboratorium 1

Michał Waluś 279695

Kwiecień 2026

1 Problem

Rozważamy TSP na zbiorach danych z https://www.math.uwaterloo.ca/tsp/world/countries.html.

W TSP zakładamy graf pełny $G = (V, E)$, gdzie $|V| = n$. Wszystkich możliwych permutacji wierzchołków mamy $n!$, ale w cyklu Hamiltona nie ma znaczenia, od którego wierzchołka zaczynamy, więc każdemu cyklu odpowiada n permutacji. Oznacza to, że mamy $(n-1)!$ możliwych cykli.

Do rozwiązania problemu korzystamy z metody *Local Search* z otoczeniem *invert*. $invert(\pi, i, j)$ polega na odwróceniu kolejności wierzchołków o indeksach od i do j w permutacji π. Łatwo zauważyć, że dla $i = j$ nic się nie zmienia. Dodatkowo z perspektywy rozwiązania TSP, $invert(\pi, 1, k)$ daje takie samo rozwiązanie jak $invert(\pi, k+1, n)$, ponieważ w obu przypadkach usuwamy krawędzie $\{1, n\}$ i $\{k, k+1\}$, a dodajemy $\{1, k+1\}$ oraz $\{k, n\}$. Oznacza to, że nie musimy rozważać n jako żaden z parametrów. Rozważamy więc następujące wartości:

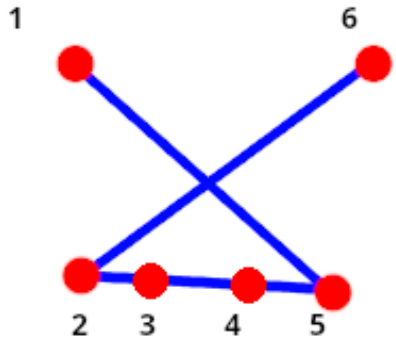
Dla $i = 1$: $j = 2, 3, \ldots, n-1$.
Dla $i = 2$: $j = 3, 4, \ldots, n-1$.
$\vdots$
Dla $i = k$: $j = k+1, k+2, \ldots, n-1$.
$\vdots$
Dla $i = n-2$: $j = n-1$.

Zatem łącznie dla każdego cyklu jego otoczenie *invert* zawiera $(n-2) + (n-3) + \cdots + 2 + 1 = \frac{(n-1)(n-2)}{2}$ innych rozwiązań.

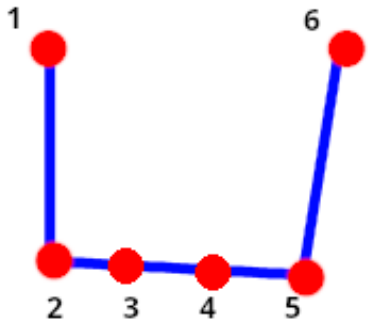
Ponieważ modyfikacja permutacji za pomocą *invert* odwraca jeden ciągły fragment (usuwa dwie krawędzie i dodaje dwie nowe między czwórką wierzchołków), to usuwa też przecięcia. Widać to na Rysunku 1.

Dla danych zbiorów danych, odległość jest obliczana jako odległość Euklidesowa zaokrąglona do najbliższej liczby całkowitej. Skutkuje to tym że w zwróconych rozwiązaniach mogą występować przecięcia. Jako przykład rozważmy cztery wierzchołki o współrzędnych: $(0, 0), (0, 1), (1, 0), (1, 1)$. Minimalny cykl na nich tworzy kwadrat o obwodzie cztery. Zauważmy jednak że przy wyżej opinaych zaokrągleniach, przekątna tego kwadratu ma długość $\sqrt{2} = 1.41... \approx 1$, zatem cykl $(0, 1) \to (0, 0) \to (1, 1) \to (1, 0)$ również ma długość 4, więc według warunków naszego zadania jest optymalny mimo przecięcia. Oba cykle zostały przedstawione na rysunku 2.

Wyniki wszystkich zadań zostały przedstawione w tabelach 1–3.

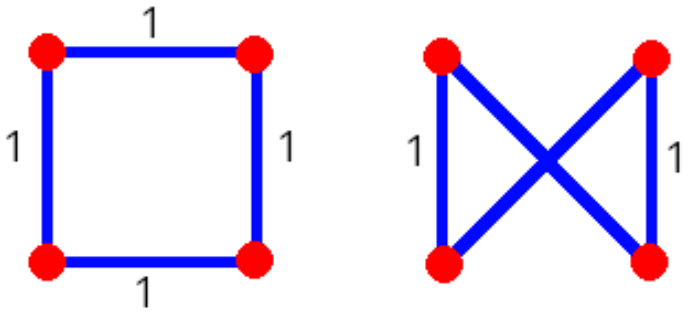


(a) Przecięcie



(b) Krótsza trasa

Rysunek 1: Eliminacja przecięcia



Rysunek 2: Optymalne cycle

2 Zadanie 1

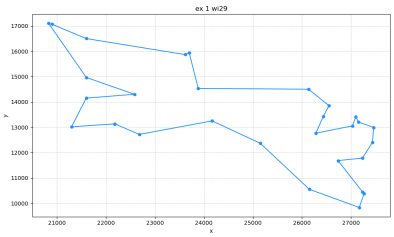
Celem zadania jest wykonanie algorytmu *Local Search* dla n losowych permutacji, dla każdej wykonując go, przeszukując całe otoczenie *invert*, aż przestaniemy znajdować lepsze rozwiązania. Ponieważ przeszukiwanie całego otoczenia jest bardzo czasochłonne, nie jest możliwe w sensownym czasie wykonanie algorytmu dla n permutacji, gdy $n > 1000$. Zatem dla $n < 1000$ rozwiązanie zostało wyznaczone dla n permutacji, dla $n \in [1000, 2000)$ dla 60, a dla większych dla 6.

Wykresy uzyskanych rozwiązań dla małych danych przedstawione są na rysunku 3.

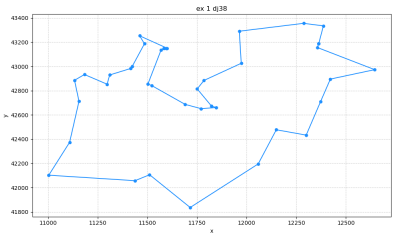
3 Zadanie 2

Celem zadania jest przyspieszenie wykonywania wolnego algorytmu *Local Search* i sprawdzanie dla n losowych sąsiadów zamiast całego sąsiedztwa.

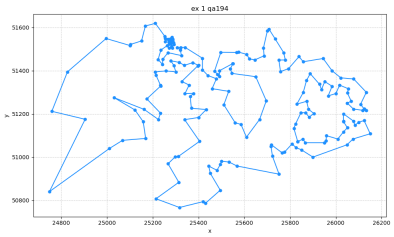
Wykresy uzyskanych rozwiązań dla małych danych przedstawione są na rysunku 4.
Widzimy, że uzyskane rozwiązania są znacznie gorsze niż w zadaniu pierwszym i nawet dla Sahary Zachodniej zawierają przecięcia krawędzi.



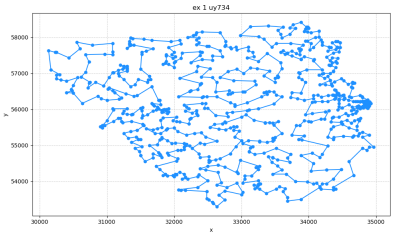
(a) Sahara Zachodnia



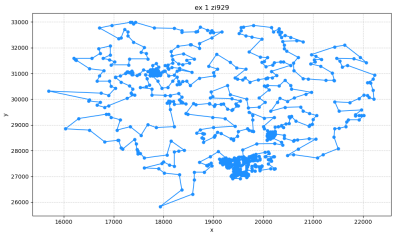
(b) Dżibuti



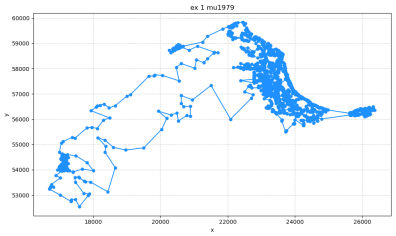
(c) Katar



(d) Urugwaj



(e) Zimbabwe



(f) Oman

Rysunek 3: Uzyskane rozwiązania w Zadaniu 1.

4 Zadanie 3

Celem zadania było wykonanie algorytmu *Local Search* dla lepszego punktu startowego niż losowa permutacja, tzn. dla rozwiązania uzyskanego z przeszukiwania w głąb na minimalnym drzewie rozpinającym. Wierzchołek startowy przeszukiwania był wybierany losowo $\lceil \sqrt{n} \rceil$ razy (dla małych danych zwiększyłem to do n by uzyskać lepsze wyniki).

Do wyznaczenia minimalnego drzewa rozpinającego wykorzystujemy algorytm Prima:

- Rozpocznij konstrukcję drzewa jako losowy wierzchołek z grafu.
- Do drzewa dołóż krawędź o najmniejszej wadze spośród wszystkich tych łączących drzewo z pozostałymi wierzchołkami.
- Powtarzaj krok drugi, dopóki wszystkie wierzchołki nie należą do drzewa.

Wykresy uzyskanych rozwiązań dla małych danych przedstawione są na rysunku 5.

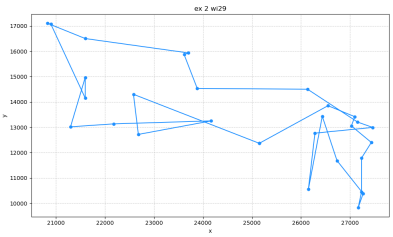
5 Wyniki

Wyniki w tabeli 2 zostały przedstawione po obcięciu części dziesiętnej, a w tabeli 3 zaokrąglone do 4 cyfr znaczących.

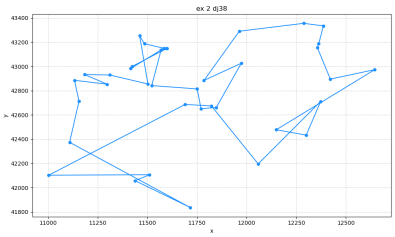
W tabeli pierwszej widzimy, że uzyskane rozwiązania w zadaniu drugim są wyraźnie gorsze niż dla zadań 1 i 3, co zgadza się z oczekiwaniami. Co ciekawe, dla dwóch najmniejszych zbiorów danych udało nam się uzyskać rozwiązania optymalne, aczkolwiek dla większych zbiorów jest wyraźna różnica. Widoczne jest jednak, że rozwiązania uzyskane z MST są lepsze od tych z losowej permutacji.

Tabela druga, zawierająca średnie rozwiązania i rozmiar MST, nie zawiera żadnych niespodziewanych rezultatów. Widzimy jednak, że rozmiary MST są mniejsze od rozwiązań optymalnych.

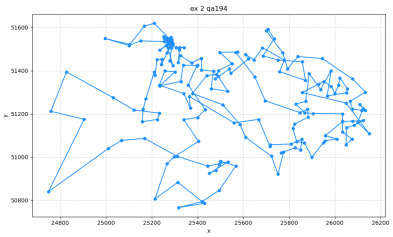
Tabela trzecia, zawierająca liczby kroków, obrazuje dwie rzeczy:



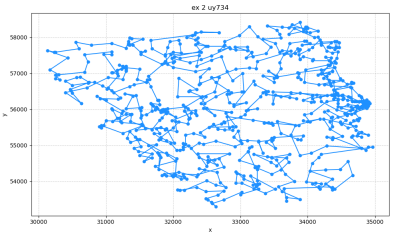
(a) Sahara Zachodnia



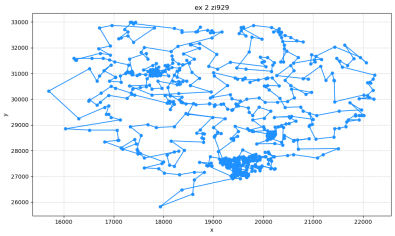
(b) Dżibuti



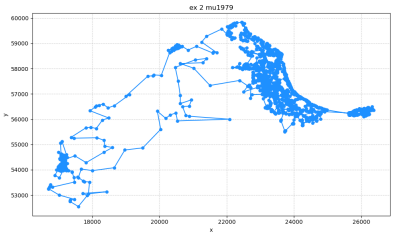
(c) Katar



(d) Urugwaj



(e) Zimbabwe



(f) Oman

Rysunek 4: Uzyskane rozwiązania w Zadaniu 2.

Dane	Zadanie 1	Zadanie 2	Zadanie 3	Optimum
Sahara Zachodnia ($n = 29$)	27603	33504	27603	27603
Dżibuti ($n = 38$)	6656	7892	6656	6656
Katar ($n = 194$)	9745	11842	9511	9352
Urugwaj ($n = 734$)	84162	131189	84249	79114
Zimbabwe ($n = 929$)	100298	123055	100817	95345
Oman ($n = 1979$)	92440	145202	90811	86891
Kanada ($n = 4663$)	1414362	2291819	1397234	1290319
Tanzania ($n = 6117$)	432213	765313	430891	394718
Egipt ($n = 7146$)	187823	303652	185423	172387
Irlandia ($n = 8246$)	233954	454527	231547	206171

Tabela 1: Najlepsze Rozwiązanie i Optymalne

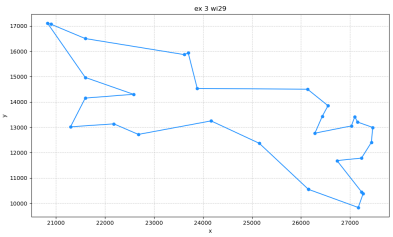
1. W zadaniu drugim, mimo tak samo "złego"rozwiązania początkowego jak w zadaniu pierwszym, program wykonuje mniej kroków poprawy (wyjątkiem jest Zimbabwe).

2. Rozpoczęcie z MST wyraźnie zmniejszyło liczbę kroków potrzebnych do zakończenia algorytmu (o około 40%).

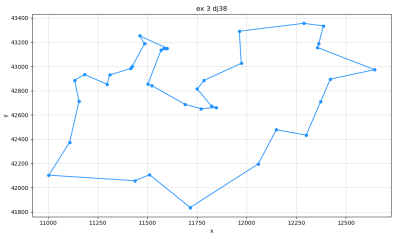
6 Wnioski

Algorytm *Local Search* daje wyraźnie lepsze wyniki od tych uzyskanych poprzez zwykłe losowanie permutacji, co jest całkowicie oczekiwanym rezultatem, jednak czas działania algorytmu jest spory, a dla zbiorów danych mających powyżej 100 wierzchołków uzyskane rozwiązania są wyraźnie (o około 10%) gorsze od optimum. Próba przyspieszenia algorytmu poprzez sprawdzanie tylko n sąsiadów w każdej iteracji prowadzi do uzyskania znacznych gorszych rozwiązań, co nie jest dobrym rezultatem.

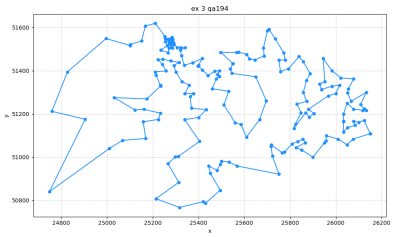
Jakość rozwiązania poprawia wybranie lepszego rozwiązania początkowego (np. powstałego z MST), jednak algorytm dalej jest bardzo wolny (aczkolwiek zmniejszenie liczby kroków powoduje minimalne przyspieszenie), a wyniki są odległe od optimum.



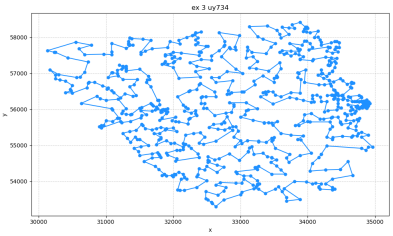
(a) Sahara Zachodnia



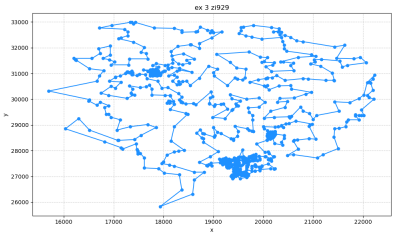
(b) Dżibuti



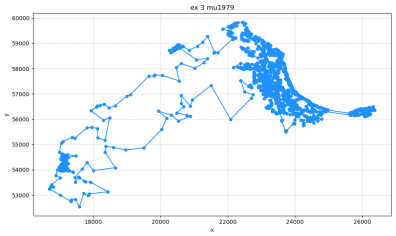
(c) Katar



(d) Urugwaj



(e) Zimbabwe



(f) Oman

Rysunek 5: Uzyskane rozwiązania w Zadaniu 3.

Dane	Zadanie 1	Zadanie 2	Zadanie 3	MST
Sahara Zachodnia ($n = 29$)	28494	36026	28688	21753
Dżibuti ($n = 38$)	6947	9734	6759	5828
Katar ($n = 194$)	10138	15607	10008	8026
Urugwaj ($n = 734$)	86923	164523	86007	69873
Zimbabwe ($n = 929$)	104227	150475	104276	82863
Oman ($n = 1979$)	94372	175357	92362	74109
Kanada ($n = 4663$)	1478548	2683538	1443468	1118421
Tanzania ($n = 6117$)	452156	870521	480983	348447
Egipt ($n = 7146$)	191902	362051	190653	147514
Irlandia ($n = 8246$)	237263	522731	234324	185370

Tabela 2: Średnie Rozwiązanie i MST

Lepszym sposobem przyspieszenia algorytmu mogłoby być zaczynanie kolejnej iteracji po znalezieniu pierwszego sąsiada z lepszym rozwiązaniem, zamiast sprawdzania wszystkich, jednak prawdopodobnie prowadziłoby to do gorszych rozwiązań (prawdopodobnie lepszych niż dla MST).

W celu uzyskania lepszego rozwiązania po zakończeniu pracy tego algorytmu moglibyśmy spróbować zastosować na końcowej permutacji kolejny algorytm *Local Search*, ale z innym wyborem sąsiedztwa (np. poprzez zamianę dwóch wierzchołków).

Dane	Zadanie 1	Zadanie 2	Zadanie 3
Sahara Zachodnia ($n = 29$)	24.79	22.66	16.76
Dżibuti ($n = 38$)	34.68	30.95	24.08
Katar ($n = 194$)	213.0	199.6	152.8
Urugwaj ($n = 734$)	925.0	832.2	584.7
Zimbabwe ($n = 929$)	1176	1254	755.8
Oman ($n = 1979$)	2789	2630	1675
Kanada ($n = 4663$)	6774	6640	4224
Tanzania ($n = 6117$)	8892	8663	5613
Egipt ($n = 7146$)	9987	9850	6623
Irlandia ($n = 8246$)	11460	10970	7124

Tabela 3: Średnia Liczba kroków